



Test Architecture Optimization for Post-bond Test and Pre-bond Tests of 3D SoCs Using TAM Reuse

Surajit Kumar Roy and Chandan Giri

Department of Information Technology, Indian Institute of Engineering Science and Technology, Shibpur, Howrah, West Bengal, India

ABSTRACT

Three-dimensional (3D) integration based on through-silicon-via (TSV) is an emerging technology. It provides reduced interconnection length, heterogeneous integration, higher performance, and bandwidth. However, 3D design poses several challenges and testing of 3D integrated circuits (ICs) is a key challenge. So, effective test mechanisms are necessary to test this new generation chip. In this paper, we address test architecture optimization and test schedule for 3D System-on-Chip (SoC). A novel heuristic is suggested to minimize the post-bond test time of 3D SoC under the constraints of TSV and power consumption. Also, the total test time is minimized by considering the routing cost of test access mechanisms (TAMs). We minimize the TAM wire length by sharing TAM segments in pre-bond tests and post-bond test. Simulations are carried out on different ITC'02 test benchmarks that demonstrate the efficiency of the proposed method.

KEYWORDS

Keywords: Post-bond test; Pre-bond test; SoC; TAM; TSVs

1. INTRODUCTION

Three-dimensional (3D) integrated circuits (ICs) have gained considerable interest among the researchers to conquer the interconnect bottleneck in CMOS scaling. In a 3D IC, several dies are stacked one above another vertically. Connections between two layers are provided by vertical interconnects, known as through-silicon-vias (TSVs) [1]. A conceptual view of a 3D IC is presented in Figure 1. Here, the 3D IC contains two layers where each layer includes various design blocks and layers are interconnected by TSVs. Three-dimensional integration offers many benefits over 2D integration [2, 3] like higher performance and reduced power consumption due to the reduced interconnect length, larger bandwidth, heterogeneous integration, and small footprint. Although 3D IC has many benefits, it suffers from several test challenges [4]. Three-dimensional integration can be explored to System-on-Chip (SoC) design, which results in 3D SoC. In a 3DSoC, the cores are placed at several dies. Modular test approach provides testing of cores in 3D SoC using test access mechanism (TAM). The TAM transports test data to the core from the test source and collects test responses from the core to the test sink. It is known that all the test pins are placed at the lower layer of 3D SoC. Hence, TAMs must be extended through TSVs to the upper layers of 3D SoC. But small chip area restricts the number of TSVs and hence, TSV constraint is to be considered during test scheduling. Maximum power limit should also be taken into consideration to prevent

chip overheating. Therefore, TAM optimization for 3D IC should consider constraints such as the number of available TSVs, placement of cores in different dies of 3D SoC, and maximum power consumption. It is also known that test architecture optimization to reduce the test time of SoC is an NP-Hard problem [5]. So, the effective test architecture optimization technique is required for 3D SoCs. This inspires to propose a test architecture optimization method for minimizing the test time of 3D SoC for the final complete stack (post-bond test).

The test flow of 3D SoC is different from 2D SoC [6]. In 3D SoC, the first pre-bond test is conducted for the individual die before stacking to get known good die (KGD). Then KGDs are stacked and finally, post-bond test is carried out for testing the complete stack. Intermediate tests are required for partial stacks. Here, we consider only pre-bond and post-bond tests. The sum of the pre-bond time and post-bond test time is considered as the total test time of the 3D SoC. Hence, the test architecture optimization should consider pre-bond tests and post-bond tests. If we use different TAMs for pre-bond tests and post-bond tests, then there will be a large TAM routing cost. Therefore, our main objective is to reuse pre-bond TAMs and post-bond TAMs for minimizing the total test time of 3D SoC.

Various works on 3D IC testing have been addressed in the literature. Authors in [7, 8] have tried to design

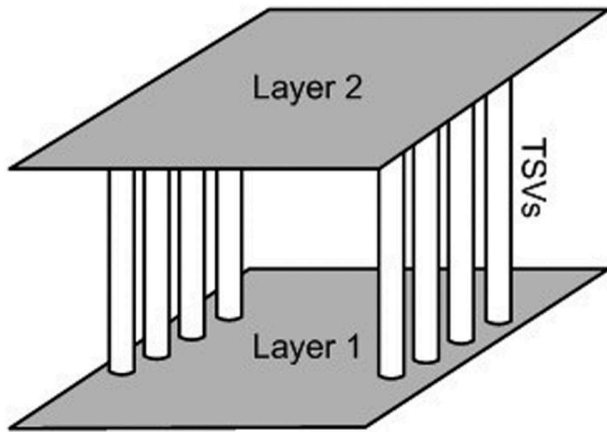


Figure 1: Example of 3D SoC

1500-style test wrapper for 3D SoCs based on TSVs. An integer linear programming (ILP)-based method to minimize the test time of 3D SoC using TSV constraint is explored in [9]. Here, the TAM wires are divided into different test buses and cores are assigned to test buses for minimizing test time. In [10], the authors have addressed a test time optimization technique that considers TAM wire length constraint for both pre-bond and post-bond tests. But the method does not consider TSV constraint. In [11], a die-level wrapper for pre-bond and post-bond tests is presented. An ILP-based optimization technique to minimize test time for complete stack of 3D SIC has been described in [12, 13]. Here, authors consider test architecture optimization for both hard and soft dies. Multiple test insertions for testing the partial stack are not considered in these works. Noia *et al.* [14] address this issue and propose an optimization method for minimizing the test time of a 3DSIC, considering multiple test insertions during bonding and final complete stack. A simulated annealing (SA)-based method [15] is developed to minimize weighted test cost under the constraints on test pin width in pre-bond and post-bond tests by sharing pre-bond and post-bond TAMs. However, this technique cannot be explored to minimize the final post-bond test time using TSV constraint. There are some works on pre-bond testing and TSV testing [16–18]. A session-based approach to minimize the post-bond test time is proposed in [19, 20]. In this paper, we have extended the works in [19, 20] for minimizing the total test time of 3D SoCs so that pre-bond and post-bond TAMs are reused as much as possible.

In this paper, we have proposed efficient heuristic to minimize the total test time of 3D SoCs so that pre-bond and post-bond TAMs are reused as much as possible. Overall, the paper contributions are as follows:

- (a) Here, a session-based approach is proposed to minimize the total test time of 3D SoCs such that sharing of pre-bond and post-bond TAMs is maximized. Proposed method shows improvement in total test time of 3D SoC than [10]. In [10], total TAM width is partitioned into several fixed width TAMs and cores are assigned to TAMs. Proposed method represents test schedule as a collection of sessions using flexible width TAM. So, TAM wires are efficiently utilized for minimizing test time.
- (b) Proposed scheme can be used to minimize the final post-bond test time together with TSV limit and power constraint.

The sequel of the paper is as follows. Section 2 describes the problem addressed in this paper. Section 3 presents proposed algorithm for post-bond test. Section 4 describes the method for pre-bond tests. Section 5 illustrates the TSV and power constraints. Section 6 presents experimental results. Finally, Section 7 concludes the paper.

2. PROBLEM FORMULATION

The test time of a 3D SoC is considered as the sum of pre-bond test time and post-bond test time. Pre-bond tests are performed for testing the individual dies before stacking. In general, a number of cores are tested simultaneously for minimizing test time. Pre-bond test minimizes the test time by considering the cores of a given layer, whereas post-bond test minimizes the test time by considering all the cores of 3D SoC. Post-bond test time will be longer if the pre-bond test schedules are utilized in the post-bond test schedule. Also, the pre-bond test time will be longer if pre-bond tests are performed using the post-bond test schedule. Because the parallel tested cores of the post-bond schedule may be placed at the different dies of 3D SoC. So some TAMs may be unused for the pre-bond test of a single layer using the post-bond schedule.

Test pads are required for the pre-bond test. It is essential to consider the pre-bond test pin constraint during test planning as test pads consume a significant area compared to TSVs. Different test schedules for pre-bond tests and post-bond test may be used to tackle this problem. So, separate dedicated TAMs need to be used during pre-bond tests and post-bond test. However, this will increase the TAM routing cost. So, reuse of TAMs for pre-bond and post-bond tests can be a good solution. Design-for-testability (DfT) circuits are required for implementing this reuse facility. This work tries to design an effective TAM architecture to maximize the reuse of pre-bond and post-bond TAMs so that the total test time of the 3D SoC

will be minimized. Now, the problem formally can be defined as follows:

Given a 3D SoC with (i) n number of cores, (ii) maximum available TAM width (W_{max}), (iii) location of the cores in different layers, (iv) test time of the cores for different TAM widths, find the test schedule of the 3D SoC to minimize post-bond test time. Also minimize the total test time of the 3D SoC considering the maximum TAM reuse for pre-bond tests and post-bond test.

The total test time T_{total} and TAM wire length $Wire_Length_{total}$ of 3D SoC are represented as follows:

$$T_{total} = T_{post} + T_{pre} \quad (1)$$

$$Wire_Length_{total} = Wire_Length_{post} + Wire_Length_{pre} \quad (2)$$

Here, T_{post} and $Wire_Length_{post}$ represent test time and TAM wire length for the post-bond test, respectively. T_{pre} and $Wire_Length_{pre}$ represent test time and TAM wire length for all the layers of 3D SoC in the pre-bond test, respectively.

To solve this problem, a session-based method is proposed for minimization of test time of the final stack. The test schedule is presented as a set of test sessions. TAM wire length determines the TAM routing cost. This is obtained by calculating the TAM wire length for connecting cores of several sessions of a schedule. In this work, pre-bond test method finds the test schedules for pre-bond tests depending on post-bond sessions and post-bond TAMs are reused in pre-bond TAM routing. Here, 'flexible width' TestBus architecture is considered [21]. Figure 2 shows the TestBus architecture with four cores. Cores connected to same TAM wires are to be tested serially, but cores connected to different TAM buses can be tested simultaneously depending on the test schedule. Test wrapper is required for testing a core. This paper considers *Design_wrapper* algorithm [5] for the wrapper design of a core. The following section discusses about the post-bond testing approach.

3. POST-BOND TESTING

The proposed post-bond test method (*post_schedule()*) creates a number of test sessions using the maximum available TAM width. A session is defined as a collection of cores that are tested in parallel and the test time of the session is the maximum test time of all the cores of the corresponding session. The algorithm *post_schedule* is demonstrated in Figure 3. At first, the procedure *Sessions_schedule()* (shown in Figure 4) is used

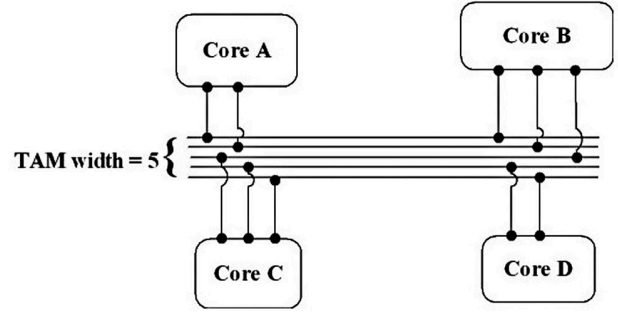


Figure 2: TestBus architecture

Algorithm 1: Post_schedule ()

Input: C: Cores of SoC with test time specification
Output: Post-bond test time.

1. $T_{post} = 0$;
2. **While** ($C \neq \text{Null}$) {
3. *Sessions_schedule()*;
4. $P = \text{Cores in first session}$;
5. Find the test time (*first_session_time*) of first session.
6. $T_{post} = T_{post} + \text{first_session_time}$.
7. $C = C - P$;
- }

Figure 3: Proposed algorithm for post-bond test

to find a schedule. Now the first session of the schedule is recorded. Then the scheduled cores and test time of the first session are obtained. Now we find out the cores except the cores of the first session and these cores are to be scheduled in the next session(s). Similarly, the *Sessions_schedule()* procedure is followed to the cores still to be scheduled and every time the test time of the first session is recorded. Overall, the post-bond test time is obtained by summing up all the sessions test times. Overall, the post-bond test time is obtained by summing up all the sessions test times.

Minimization of the test time of the first session is obtained by the procedure *Sessions_schedule()* (presented in Figure 4). The session formation is started by choosing the largest core (core having highest test time for W_{max}) and maximum available TAM width W_j ($W_j \leq W_{max}$) is assigned to that core. It is possible if the test time of the corresponding core with respect to W_j and W_{max} are the same (because of Pareto-optimal point [21]). Test time of the session is the test time of the largest core. Once all the TAMs are allocated to the cores of an SoC, it creates a session. New session will be formed for the remaining

Algorithm 2: Sessions_schedule()**Input:** $S \rightarrow$ set of cores; $W_{\max} \rightarrow$ Maximum available TAM width.**Output:** test schedule.

```

1.  $T_p = \text{sum of test time of all cores using } W_{\max}$ ;
2.  $\text{schedule} = \text{serial testing of all cores}$ ;
3.  $\text{flag} = 1; \text{first} = 1$ ;
4. Do {
5.    $T = 0; C = S$ ;
6.    $\text{sn} = 0$ ; //  $\text{sn}$  is used for storing session numbers
7.   While ( $C \neq \text{Null}$ ) {
8.      $W_a = W_{\max}; \text{sn} += 1$ ;
9.      $\text{SC}_{\text{sn}} = \text{Null}$ ; //  $\text{SC}_x \rightarrow$  set of cores in session  $x$ 
10.    If ( $\text{first} = 1$  OR  $\text{sn} > 1$ ) then {
11.      Choose core  $i \in C$  such that  $T_i(W_a)$  is maximum;
12.      Find  $w_j \leq W_a$  such that  $T_i(w_j) = T_i(W_a)$ ;
13.      /*  $w_j$  is the TAM Width allocated to  $i^{\text{th}}$  core  $j$  and
14.        $T_i(w_j)$  is the test time of  $i^{\text{th}}$  core for TAM width  $w_j$  */
15.    } Else {
16.       $i = c1; w_j = w1 - 1$ ;
17.      If ( $w_x < w_j$  such that  $T_i(w_x) = T_i(w_j)$  is found) then
18.         $w_j = w_x$ ;  $\text{flag} = 0$ ; }
19.     $W_a = W_a - w_j; C = C - i; T_{\text{sn}} = T_i(w_j)$ ;
20.     $\text{SC}_{\text{sn}} = \text{SC}_{\text{sn}} \cup \{i\}$ ;
21.    If ( $\text{sn} = 1$ ) then {
22.       $w1 = w_j; c1 = i$ ; } /* storing the information of the
23.      1st core of 1st session */
24.    While ( $W_a > 0$ ) {
25.      Choose  $k \in C$ , such that  $T_k(w_m)$  is maximum;
26.      Find  $w_l \leq W_a$  such that  $T_k(w_l) = T_k(W_a)$  AND
27.       $T_k(w_l) \leq T_{\text{sn}}$ ;
28.       $W_a = W_a - w_l; C = C - k$ ;
29.       $\text{SC}_{\text{sn}} = \text{SC}_{\text{sn}} \cup \{k\}$ ;
30.       $T = T + T_{\text{sn}}$ ;
31.       $\text{new\_schedule} = \{\text{SC}_s \mid s = 1 \text{ to } \text{sn}\}$ ;
32.      If ( $T < T_p$ ) then {
33.         $T_p = T$ ;  $\text{schedule} = \text{new\_schedule}$ ; }
34.       $\text{first} = 0$ ;
35.    } While ( $\text{flag}$ );
36.  } Return ( $T_{\text{total}}, \text{schedule}$ );

```

Figure 4: Procedure *Sessions_schedule()*

cores. Otherwise, cores are included in the current session so that remaining TAMs are utilized. Then, the next core having highest test time is selected whose required TAM width is less than the available TAM width and test time of the selected core is lesser or equal to test time of the corresponding session. In this way, cores are included in the current session and the session is formed. Similarly, remaining cores are scheduled in several sessions. Overall, testing time is obtained by summing up the test time of all sessions. For minimizing the test time further, TAM width of the largest core of the first session is reduced by one and the scheduling procedure is again executed. In this way, we repeat the whole scheduling process until pare-to-optimal points for two consecutive TAMs are not same. Now details of only the first session of the iteration having minimum test time are recorded as the final scheduling for a given TAM width and returns this arrangement to the main algorithm (*Post_schedule()*).

3.1 Time Complexity

Procedure *session_schedule()* creates a single session by choosing the current largest core i (with respect to test time) and the available TAM width W_a is updated depending on TAM requirement of the corresponding largest core w_i (and $W_a = W_{\max} - w_i$). The largest core is chosen in $O(n)$ time, where n is the number of cores in an SoC. The remaining cores of the session are selected depending on W_a so that the test time of chosen core is less than or equal to the test time of current largest core. So a session is formed in $O(W_a n)$ time. Now other sessions are formed from the remaining cores. So, worst-case time complexity is $O(W_a n^2)$. Then, the method is repeated until the Pareto-optimal point for two consecutive TAM widths is not the same for the largest core of the first session. Hence, the iteration will run another w_i times and the time complexity of procedure *session_schedule()* is $O(w_i W_a n^2)$. The algorithm *post_schedule()* calls *session_schedule()* and chooses the first session until all the cores are scheduled. So, the iteration will execute n times more. Therefore, the time complexity of the algorithm is $O(w_i W_a n^3) \approx O(W_{\max}^2 n^3)$.

3.2 TAM Routing

TAM wire length for post-bond TAM routing is calculated using different post-bond sessions. Our objective is to minimize overall TAM routing cost by reusing post-bond TAM segments in pre-bond tests. We use the nearest neighbor approach to connect the cores of different sessions. Here, cores of the first session are connected to test pins, cores of the second session are connected to the cores of the first session, cores of the third session are connected to the cores of the second session, and so on. Also, the last session is connected to test pins. Hence, TAM wire length for a given session SC_{sn} is the distances between the cores of SC_{sn} and cores of $\text{SC}_{\text{sn}-1}$. A core of SC_{sn} is connected to the nearest core of $\text{SC}_{\text{sn}-1}$. TAM wire length WL_{sn} for a session SC_{sn} is calculated as

$$\text{WL}_{\text{sn}} = \sum D_{i,j} \times \text{TAM}_i \quad (3)$$

Here, $D_{i,j}$ and TAM_i represent the distance between the i^{th} core of session SC_{sn} and j^{th} core of session $\text{SC}_{\text{sn}-1}$, and TAM width allocated to the i^{th} core, respectively. TAM wire length for the post-bond test ($\text{Wire_Length}_{\text{post}}$) is the sum of wire lengths for all the sessions.

3.3 Illustrative Example

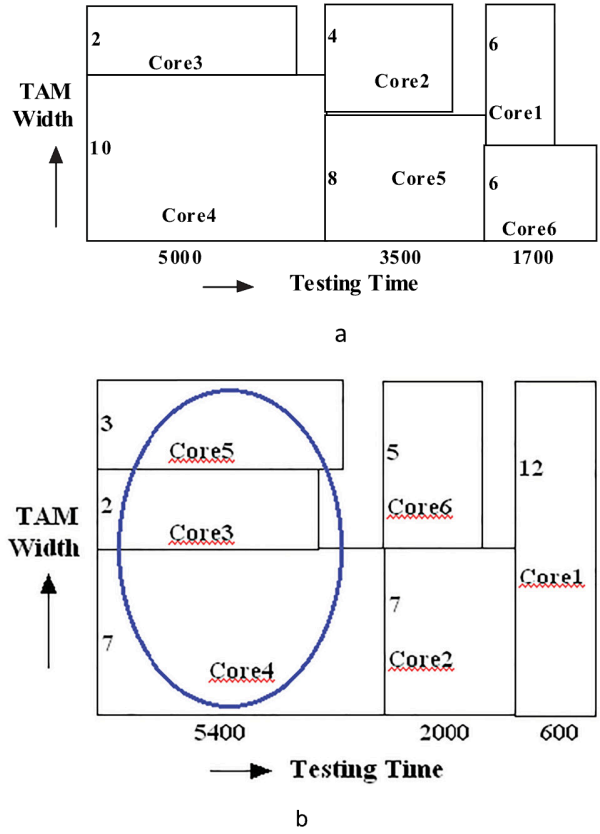
Consider a 3D SoC having six cores as *Core1*, *Core2*, *Core3*, *Core4*, *Core5*, and *Core6*. $W_{\max}$ is considered as

Table 1: Test time of cores of example SoC for different TAMs

TAM width	Core1	Core2	Core3	Core4	Core5	Core6
1	3300	4850	5600	8500	6700	4490
2	2000	4200	4500	8300	6100	3560
3	1500	3970	4500	7940	5600	2500
4	1100	2560	4300	7500	4230	2100
5	980	2300	4050	6580	4130	1850
6	760	2300	3330	6200	4130	1700
7	760	2000	3210	5400	3790	1700
8	760	2000	2380	5400	3500	1700
9	700	2000	2100	5200	3500	1700
10	650	2000	2100	5000	3500	1700
11	650	2000	2100	5000	3500	1700
12	600	2000	1870	5000	3500	1700

12. Test time of the cores for different TAM widths are shown in Table 1. At first algorithm, *Post_schedule* will call the procedure *session_schedule()*. Now, the formation of sessions is described using *session_schedule()*. According to this procedure, Core4 is chosen first because its test time is the maximum among all the cores for TAM width 12. But the test time of Core4 for TAM width 10 and 12 are the same. Hence, Core4 uses TAM width 10 and the procedure will attempt to test the non-tested cores simultaneously with Core4 using remaining TAM width 2. Now, a larger core is selected such that its test time for TAM width 2 is less than or equal to the test time of Core4. Here, Core3 is considered as its test time is the maximum for TAM width 2 and Core4 and Core3 are tested simultaneously. Hence, a session is formed by Core4 and Core3. Then Core5 is considered because it is the current largest core. TAM width 8 is allocated to Core5 as the test time of Core5 for TAM width 12 and TAM width 8 are same. Next, Core2 is selected and tested with Core5 because Core2 is the next largest core for remaining TAM width 4 and test time of Core2 for TAM width 4 is lesser or equal to Core5. This is the second session. Similarly, the third session is formed and core Core6 and Core1 are scheduled. Figure 5(a) shows the schedule of all six cores and the overall test time is calculated as 10,200 (i.e. $5000 + 3500 + 1700$) clock cycles which is less than the time of serial testing.

For further reduction of the test time, TAM width of the first core of the first session is reduced by one until it reaches Pareto-optimal point. Now, TAM width 9 is allocated to Core4 and *session_schedule()* is again executed. Here, all the scheduling arrangements are not shown in the figure. Again TAM width of Core4 is reduced by one and finally TAM width 7 is allocated to core c4 as test time of Core4 for TAM width 7 and 8 are the same. Similarly, the remaining cores are scheduled and final arrangement of cores is shown in Figure 5(b). The post-bond test time is 8000 (i.e. $5400 + 2000 + 600$) clock cycles. Since

**Figure 5:** Formation of first session ((a) initial; (b) final) for TAM width 12 using *session_schedule()*

this test time is the minimum among all the scheduling arrangements, the first session of this arrangement (marked as circle in Figure 5(b)) is considered as the first session of the final schedule for the given 3D SoC and returned to the main algorithm with the corresponding test time of the session.

Now the *Post_schedule()* will run *session_schedule()* again for the remaining unscheduled cores, i.e. Core1, Core2 and Core6. In a similar fashion, session is formed and the remaining cores are scheduled in a single session, as shown in Figure 6(a). This session is considered as the second session of the final arrangement. The final arrangement is obtained after scheduling all the cores and shown in Figure 6(b). Test time of this arrangement is 7700 (i.e. $5400 + 2300$) clock cycles.

4. PRE-BOND TEST

The proposed pre-bond test algorithm is described in Figure 7. The algorithm uses a session-based heuristic to find the pre-bond sessions for each die of 3D SoC using the post-bond schedule and TAM routing. The algorithm first creates partial sessions corresponding to

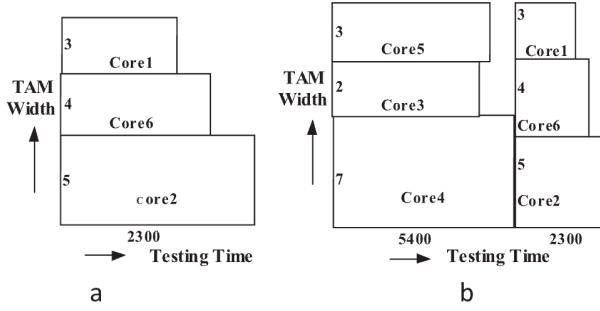


Figure 6: Formation of second session (a) and final arrangement of cores for TAM width 12 (b)

each post-bond session. A partial session is a part of the post-bond session for the cores with layer number L , i.e. a partial session contains the cores with layer number L from a post-bond session. A partial session will be empty if no core of the layer number L is included in the corresponding post-bond session. Now different pre-bond sessions will be created depending on these partial sessions. The algorithm starts with the creation of the first pre-bond session. To create the first session S_1 , the algorithm chooses cores from the first partial session and includes these cores in S_1 . Here, the TAM segments for the first session are shared in pre-bond test and post-bond test. Now if the test pins requirement of S_1 is less than the maximum available test pins, then other cores are to be included in this session. For this, the algorithm chooses the cores from remaining partial sessions except last such that the chosen core is not connected to any core of S_1 in post-bond TAM routing. If the first partial session is empty (i.e. no core of layer number L is included in the first post-bond session), then S_1 is formed by choosing cores from remaining partial sessions except last such that no two cores of S_1 are connected for post-bond TAM routing and TAM width is available. In this way, first pre-bond session S_1 is created.

After the creation of the first session, the next sessions will be generated depending on the last created session. To create the next session S_i , first we choose the cores from the partial sessions except last such that chosen cores are connected to any core of the previous session S_{i-1} in post-bond TAM routing. Here, TAM segments between the cores of S_{i-1} and S_i are shared for pre-bond and post-bond test. Similarly, remaining pre-bond sessions are formed. Now, the last pre-bond session is created from the cores of the last partial session to reuse TAM segments of the last post-bond session. Now, the last two sessions may be merged into a single session depending on the availability of TAM width.

Algorithm 3: Pre_schedule()

Input: Post-bond schedule and TAM routing, Set of cores C for a given layer number L , maximum available pre-bond test pins $W_{\max}$

Output: Pre-bond test time for layer number L .

```

1. For ( $i = 1$  to  $s_n$ )
    /* creation of partial sessions from post-bond sessions */
2.  $SC_i =$  cores with layer number  $L$  from post-bond session  $i$ ;
3. If ( $SC_1 \neq \emptyset$ ) then
    /* cores of layer no.  $L$  are included in 1st post-bond session */
4. { Include the cores of  $SC_1$  into  $S_1$ ;
5.  $W_a = W_a - \sum w_i$ ; /*  $W_a$  and  $w_i$  represent available pre-bond
    test pins and TAM width for  $i^{\text{th}}$  core respectively */
6. While ( $W_a > 0$ )
    /* cores from remaining partial sessions except last are chosen */
7. Chose core from  $SC_2$  to  $SC_{s_n-1}$  and include into  $S_1$  such
    that chosen core is not connected to any core of  $S_1$  in post-
    bond TAM routing;
8. }
9. Else {
10. While ( $W_a > 0$ )
11. Create  $S_1$  by choosing cores from  $SC_2$  to  $SC_{s_n-1}$  such that
    no two core of  $S_1$  are connected in post-bond
    TAM routing; }
12.  $C = C - S_1$ ;  $S_1 = S_1$ ;
13. While ( $C \neq \emptyset$ ) {
14.  $W_a = W_{\max}$ ;
15. Create  $S_{i++}$  by choosing cores from  $SC_2$  to  $SC_{s_n-1}$  such that
    chosen cores are connected to any core of  $S_1$  in
    post-bond TAM routing;
16. While ( $W_a > 0$ )
17. { Chose core from  $SC_2$  to  $SC_{s_n-1}$  and include into  $S_{i++}$ 
    such that chosen core is not connected to any core
    of  $S_{i++}$  in post-bond TAM routing; }
18.  $S_i = S_{i++}$ ; }
19. If ( $SC_{s_n} \neq \emptyset$ ) then
20. Include the cores of  $SC_{s_n}$  into  $S_i$ ;
21. If ( $W_{\max} \geq$  sum of test pins for  $S_{i-1}$  and  $S_i$ ) then
22. Merge  $S_{i-1}$  and  $S_i$  into a single session;
23. For each sessions  $S_i$ 
24. Apply the procedure readjust();
25.  $T_{\text{pre}_L} =$  sum of test time of all the sessions;
    /*  $T_{\text{pre}_L}$  represents pre-bond test time for layer  $L$  */

```

Figure 7: Proposed algorithm for pre-bond test

Allocated TAM width of a core is the same as the post-bond TAM width of that core. A post-bond session considers all the cores of 3D SoC during session formation. But the pre-bond test is conducted for an individual layer. Hence, to reduce test time readjustment of TAM width is required for the cores of a pre-bond session. The procedure *readjust()* finds out the actual TAM width for the cores of a session. First, the procedure allocates 1 bit TAM to each core of a session. Now every time, the procedure finds out the cores with highest and second highest test time and allocates TAM width to the core with highest test time such that its test time is less than or equal to

second highest test time. This will continue until TAMs are available. The test time of the session is the maximum test time of all the cores of the session. Figure 8 describes the the procedure *readjust()*.

5. TSV ESTIMATION AND POWER CONSTRAINT

The cores of 3D SoC are located at different layers. It is assumed that the bottom layer of 3D SoC is layer 0, followed by then layer 1, layer 2, and so on. No TSV is required for the cores at the bottom layer. Number of required TSVs for testing the i th core at L_i layer using w_i TAM is $2 \times L_i w_i$. According to the algorithm *Post_schedule*, cores of a session are tested in parallel and several sessions are serially tested. So, the TSV requirement of a given session is determined as

$$\sum 2 \times L_i w_i | i \in \text{cores of a session}$$

The total number of required TSVs for testing the 3D SoC is the maximum TSV requirement of all the sessions.

We can include the TSV constraint as the maximum number of available TSV ($TSV_{\max}$) in the test schedule. In a session, only one core is included at a time. After inclusion the core, TSV requirement of the session is evaluated and TSV constraint is checked. The TSV constraint is satisfied if the number of required TSVs of the session is less than or equal to $TSV_{\max}$.

If peak power during testing of each core is assumed to be available, we can solve the test scheduling problem using power constraint such that the overall test time is minimized satisfying power constraint. To evaluate power requirement, we calculate the sum of the maximum peak power of the cores of a session. We consider it as peak power of a session. If the peak power of a session is less than or equal to the maximum available power, then power constraint is satisfied. It is assumed that the peak power consumption of the core is the same during testing of the core.

6. EXPERIMENTAL RESULTS

Simulation results are presented for three SoCs from ITC'02 test benchmarks. The SoCs are p22810, p93791, and p34392. The proposed heuristics are coded in C programming language and executed on a HP workstation with Core i7 processor having 4 GB RAM. To demonstrate the effectiveness of the proposed algorithms, we use SoCs from ITC'02 benchmarks namely p22810, p93791, and p34392. Number of layers of the 3D SoCs for each case of simulation is considered as 3. Table 2 presents the

Algorithm4: readjust()

1. Allocate 1 bit TAM to each core of the session;
2. $W_{\max} = W_{\max} - \text{number of cores for this session};$
3. **While** ($W_{\max} > 0$) {
4. Chose two cores c_i and c_j with highest and second highest test time respectively;
5. Allocate TAM width to c_i until $T_i \leq T_j$;}
- /* T_i and T_j are test time of c_i and c_j respectively*/
6. $T = \max(T_i)$

Figure 8: Procedure *readjust()*

Table 2: Distribution of cores among different layers of 3D SoC

Layer no. (bottom 1, upper most 3)	SoCs		
	p22810	p93791	p34392
Layer 1	1, 2, 3, 5, 6, 8, 9, 15, 21, 26, 27	1, 2, 5, 8, 10, 11, 16, 18	1, 4, 5, 6, 12, 13, 14, 17, 19, 20, 23, 27, 29
Layer 2	4, 7, 10, 11, 17, 18, 22, 23, 25, 28	3, 4, 8, 12, 14, 15	3, 9, 10, 11, 16, 18, 21, 22, 24, 32
Layer 3	12, 13, 14, 16, 19, 20, 24	6, 7, 13, 17, 19	2, 7, 8, 15, 25, 26, 28, 30, 31

distribution of cores among different layers of 3D SoC. The cores of an SoC are placed among the three layers in such a way that the area of layers is almost balanced. The area of a core is estimated depending on the number of inputs/outputs, number of scan chains, and length of each of the scan chains. Then the placement information of the different cores for a layer of 3D SoC are obtained using an “Academic Floor-planar” tool as in [15].

Tables 3–5 describe the test time of the final post-bond test for TSV limit 80 for 3D SoCs p22810, p93791, and p34392, respectively. In each table, column 1 shows the TAM width. Columns 2 and 5–7 present the test time for our proposed method and the methods in [9 22, 23], respectively. The test time is presented in “Clock Cycles” (CC). Column 3 and 4 show TSV requirement and run time of our algorithm, respectively. It is seen from tables that the post-bond test time decreases with the increase in TAM width because in a session, a number of cores can be tested in parallel. This is consistent with the results obtained from 2D SoC [5]. For the SoC p34392, test time does not decrease after TAM width 48 showing the presence of the Pareto-optimal point [21]. The number of required TSVs to test a 3D SoC increases with the TAM width because more TAM width can be used for the cores of non-bottom layer. The test time of the proposed method is comparable with the earlier works [9 22, 23]. The tables describe that the proposed algorithm provides better results for minimizing test time

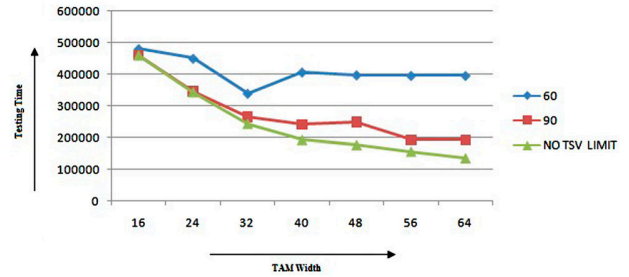
Table 3: Experimental results of p22810 for post-bond test (TSV limit = 80)

TAM width	Test time (in CC)	TSV used	Run time (s)	Test time in [9] (CC)	Test time in [22] (CC)	Test time in [23] (CC)
16	459,932	34	4.81	275,537	485,728	434,269
24	343,608	54	2.61	249,274	371,217	288,820
32	243,303	64	4.04	205,831	252,887	224,624
40	192,193	78	6.37	181,687	191,580	190,995
48	176,251	78	3.69	169,392	195,269	159,613
56	155,884	80	2.18	153,282	165,352	145,417
64	134,820	54	1.57	142,210	142,428	133,405

Table 4: Experimental results of p93791 for post-bond test (TSV limit = 80)

TAM width	Test time (CC)	TSV used	Run time (s)	Test time in [9] (CC)	Test time in [22] (CC)	Test time in [23] (CC)
16	1,796,738	64	6.69	1,779,401	1,856,606	1,753,556
24	1,208,811	76	3.46	1,197,683	1,256,380	1,175,056
32	1,060,440	80	3.36	894,463	1,108,271	882,710
40	941,346	80	2.03	778,296	1,103,105	707,262
48	624,851	80	2.35	713,347	634,441	593,119
56	575,251	80	2.25	635,095	604,257	513,755
64	475,083	80	1.97	572,342	514,652	453,742

in many cases. However, the post-bond schedule can be utilized in the pre-bond test so that the total test time of the 3D SoC will be minimized and TAM reuse will be maximized. It is also noted that the proposed heuristic can be executed in very negligible time. Figure 9 shows the variation of test time with TAM width for different number of TSVs. It is seen from this figure that test time of the SoC decreases with the increasing number of TSVs. Because a session includes more cores as number of TSVs increases, resulting in less number of sessions. So the test time is decreased. Table 6 shows the variation of the test time for SoC p93791 and p22810 with different power values. Power values of the cores for SoCs are obtained from [24]. It is seen from Table 3 that for a given value of TAM width, test time decreases with increased power budget because a set of cores can be tested in parallel.

**Figure 9: Test time of the 3D SoC p22810 for different TAM width and maximum TSV limit**

Tables 7 and 8 present the total test time for different SoCs without pre-bond test pin constraint and with pre-bond test pin constraint, respectively. We consider pre-bond TAM width as 16 for considering pre-bond test

Table 5: Experimental results of p34392 for post-bond test (TSV limit = 80)

TAM width	Test time (in CC)	TSV used	Run time (s)	Test time in [9] (CC)	Test time in [22] (CC)	Test time in [23] (CC)
16	1,020,754	52	6.67	999,543	NA	943,456
24	715,947	72	1.77	762,841	NA	647,193
32	626,094	76	3.61	665,445	NA	544,579
40	549,700	80	2.31	552,231	NA	544,579
48	544,579	36	1.45	544,579	NA	544,579
56	544,579	36	1.37	540,069	NA	544,579
64	544,579	36	2.41	534,212	NA	544,579

Table 6: Test time of the 3D SoC p93791 and p22810 for different power threshold, TAM width and TSV = 90 for layer = 3

TAM width	p93791				p22810			
	No constraints	10,000 μ W	15,000 μ W	20,000 μ W	No constraints	3000 μ W	4500 μ W	6000 μ W
16	1,796,738	1,800,363	1,800,207	1,796,738	459,932	471,229	467,968	467,168
24	1,208,811	1,255,567	1,224,639	1,224,639	343,608	375,846	373,498	366,211
32	1,060,440	1,243,557	1,218,390	1,190,785	243,303	288,515	280,304	277,742
40	1,033,275	1,116,204	1,115,993	1,102,916	192,193	285,620	243,935	238,487
48	624,851	656,817	656,601	653,771	176,251	285,620	224,207	196,986
64	475,083	644,933	644,714	641,887	134,820	285,620	218,730	192,796

Table 7: Total test time without pre-bond test pin constraint

TAM width	Total test time (CC)					% Decrease in wire length	
	No reuse	Reuse	Heuristic	% Increase in time	ln [10]	Reuse	Heuristic
P22810							
16	938,052	938,052	942,569	+0.04	1,062,281	-14.72	-18.78
24	693,071	693,071	710,198	+2.40	780,763	-13.40	-26.25
32	492,404	492,404	538,389	+9.3	627,148	-6.14	-25.5
40	422,289	422,289	424,656	+0.6	534,329	-10.0	-26.93
48	377,113	377,113	390,794	+3.63	500,868	-11.73	-14.83
56	334,749	334,749	363,333	+8.54	435,664	-14.95	-23.81
64	295,115	295,115	327,440	+10.95	421,677	-14.80	-23.49
p93791							
16	3,601,905	3,601,905	3,616,042	+0.39	3,891,988	-20.73	-24.90
24	2,439,378	2,439,378	2,437,479	-0.08	2,492,638	-16.30	-27.68
32	2,103,119	2,103,119	2,111,329	+0.39	1,857,887	-15.82	-43.88
40	1,916,477	1,916,477	1,916,583	+0.005	1,553,100	-22.53	-27.75
48	1,270,682	1,270,682	1,265,588	-0.40	1,281,244	-34.03	-30.70
56	1,179,974	1,179,974	1,183,682	+0.31	1,128,667	-33.82	-40.55
64	977,151	977,151	979,098	+0.20	988,558	-28.62	-27.74
p34392							
16	2,051,705	2,051,705	2,069,698	+0.88	2,611,379	-4.51	-22.52
24	1,434,823	1,434,823	1,435,121	+0.02	1,868,264	-3.07	-8.05
32	1,267,035	1,267,035	1,267,912	+0.07	1,498,194	-12.35	-21.96
40	1,149,831	1,149,831	1,151,113	+0.11	1,441,485	-11.21	-16.96
48	1,133,398	1,133,398	1,144,709	+0.10	1,422,943	-19.90	-22.38
56	1,132,372	1,132,372	1,127,153	-0.46	1,420,363	-20.07	-17.45
64	1,127,057	1,127,057	1,127,153	+0.01	1,413,013	-17.36	-20.83

Table 8: Total test time with pre-bond test pin constraint (number of pre-bond test pins = 16)

TAM width	Testing time					% Decrease in wire length		
	No reuse	Reuse	Heuristic	% Increase in time	ln [15]	Reuse	Heuristic	ln [15]
p22810								
16	938,052	938,052	942,569	+0.04	959,753	-14.72	-18.78	-32.3
24	821,728	821,728	847,262	+3.10	738,800	-12.4	-17.08	-25.5
32	721,423	721,423	755,140	+4.70	653,060	-6.23	-18.74	-36.9
40	670,313	670,313	692,884	+3.4	611,087	-6.48	-16.98	-43.4
48	654,371	654,371	657,380	+0.46	587,328	-8.40	-16.76	-24.9
56	634,004	634,004	640,358	+1.0	563,882	-12.53	-19.20	-37.3
64	612,940	612,940	655,584	+6.96	553,497	-10.93	-16.24	-35.8
p93791								
16	3,601,905	3,601,905	3,616,042	+0.39	3,726,714	-20.73	-24.90	-47.34
24	3,013,978	3,013,978	2,986,561	-0.90	2,791,223	-8.82	-22.51	-43.55
32	2,838,442	2,838,442	2,855,390	+0.60	2,390,750	-5.33	-18.43	-49.39
40	2,746,513	2,746,513	2,798,649	+1.90	2,153,380	-6.37	-20.99	-44.54
48	2,430,018	2,430,018	2,466,761	+1.51	1,946,263	-4.03	-21.82	-47.76
56	2,380,418	2,380,418	2,435,632	+2.32	1,842,030	-6.91	-25.21	-46.23
64	2,280,250	2,280,250	2,322,776	+1.86	1,737,586	-0.50	-31.03	-48.80
p34392								
16	2,051,705	2,051,705	2,069,698	+0.88	2,611,379	-4.51	-22.52	-48.79
24	1,746,898	1,746,898	1,809,585	+3.59	1,791,078	-7.26	-12.55	36.93
32	1,657,045	1,657,045	1,850,820	+11.69	1,595,286	-22.51	-43.42	-49.39
40	1,580,651	1,580,651	1,776,444	+12.39	1,538,795	-22.23	-38.94	-43.67
48	1,575,530	1,575,530	1,825,934	+15.57	1,567,728	-16.79	-47.36	-29.33
56	1,575,530	1,575,530	1,820,934	+15.58	1,819,571	-16.79	-47.36	-31.25
64	1,575,530	1,575,530	1,820,934	+15.58	1,758,060	-16.79	-47.36	-32.06

pin constraint. Here, we propose three types of test architecture. The first approach is denoted as *No reuse*. We apply Algorithm 1 to the 3D SoC for post-bond test of the final stack and pre-bond test of each layer. Then we find out the TAM wire length for both post-bond TAMs and pre-bond TAMs without sharing TAM segments for these two types of TAMs. In the second baseline method (known as *Reuse*), we apply Algorithm 1 for post-bond and pre-bond test as *No reuse*. Hence, the total test time of these two methods is the same. If the TAM segments

of pre-bond tests are included in post-bond TAM routing, then post-bond TAMs are reused in pre-bond tests and corresponding segments are excluded during the calculation of TAM wire length for pre-bond tests. So, the total TAM wire length of *Reuse* approach is less than *No reuse* approach. In the third approach (column 4 of Tables 7 and 8), we apply Algorithm 1 for post-bond test and Algorithm 3 for pre-bond tests. In this heuristic approach, sharing of TAMs for pre-bond and post-bond tests is maximized. Table 7 describes that the proposed

algorithm provides better results for minimizing total test time than [10]. Tables 7 and 8 also show that the proposed scheme provides a sharp decrease in wire length with small increase in total test time. The maximum wire length reduction is 43.88% for TAM width 32 of SoC p93791 without pre-bond test pin constraint, while for pre-bond test pin constraint, the maximum reduction in wire length is 47.36% using SoC p34392. The decrease in the wire length of *Reuse* approach is much less than the proposed heuristic method (shown in column 8 in Tables 7 and 8) because sharing of TAMs for pre-bond tests is maximized in our method.

7. CONCLUSION

We have introduced an efficient and effective test method for minimizing the total test time of 3D SoCs. The proposed method incorporates two session-based approaches for pre-bond tests and post-bond test. Algorithm for post-bond test includes TSV constraint as well as power constraint. Algorithm for pre-bond test includes pre-bond test pin constraint due to the limitation of the number of test pads. The proposed algorithms minimize TAM wire length as well as total test time of 3D SoCs such that sharing of TAMs for pre-bond tests and post-bond test is maximized. Experimental results are performed for ITC'02 SoC test benchmarks, which show the effectiveness of the algorithms.

ORCID

Surajit Kumar Roy  <http://orcid.org/0000-0003-3458-6874>

REFERENCES

1. H. Loh, Y. Xie, and B. Black, "Processor design in 3D die-stacking technologies," *Proc. IEEE Micro*, Vol. 27, pp. 31–48, 2007.
2. K. Banerjee, S. J. Souri, P. Kapur, and K. C. Saraswat, "3-D ICS: A novel chip design for improving deep sub-micrometer interconnect performance and system-on-chip integration," *Proc. IEEE*, Vol. 89, no. 5, pp. 602–33, 2001.
3. W. R. Davis, J. Wilson, S. Mick, J. Xu, H. Hua, C. Mineo, A. M. Sule, M. Steer, and P. D. Franzon, "Demystifying 3DICs: The pros and cons of going vertical," *IEEE Des. Test Comput.*, Vol. 22, no. 6, pp. 498–510, 2005.
4. H. S. Lee, and K. Chakrabarty, "Test challenges for 3D integrated circuits," *IEEE Design and Test of Computers, Special Issue on 3D IC Design and Test*, Vol. 5, 26–35, 2009.
5. V. Iyengar, and K. Chakraborty, "Test wrapper and test access mechanism co-optimization for system-on-chip," *Journal of Electronic Testing: Theory and Applications (JETTA)*, Vol. 18, pp. 213–30, 2002.
6. E. J. Marinissen, "Challenges in testing TSV-based 3D stacked ICs: Test Flows, test Contents, and test access," in *Proc. of IEEE Asia-Pacific Conference on circuits and Systems*, 2010, pp. 544–7.
7. S. K. Roy, S. Ghosh, H. Rahaman, and C. Giri, "Test wrapper design for 3D system-on-chip using optimized number of TSVs," *Proc. IEEE Intl. Symp. Electr. Syst. Des.*, 197–202, 2010.
8. B. Noia, K. Chakrabarty, and Y. Xie, "Test-wrapper optimization for embedded cores in TSV-based three-dimensional SOCs," in *Proc. of IEEE International Conference on Computer Design*, 2009, pp. 70–7.
9. X. Wu, Y. Chen, K. Chakrabarty, and Y. Xie, "Test access mechanism optimization for core-based three-dimensional SoCs," *Microelectron. J.*, Vol. 41, no. 10, pp. 601–15, 2010.
10. L. Jiang, L. Huang, and Q. Xu, "Test architecture design and optimization for three-dimensional SoCs," *Proc. Des. Autom. Test in Europe (DATE)*, 220–5, 2009.
11. E. J. Marinissen, J. Verbree, and M. Konijnenburg, "A structured and scalable test access architecture for TSV-based 3D stacked ICs," in *Proc. of VLSI test Symposium*, 2010, pp. 269–74.
12. B. Noia, K. Chakrabarty, and E. J. Marinissen, "Optimization methods for post-bond testing of 3D stacked ICs," *J. Electr. Test.*, Vol. 28, no. 1, pp. 102–20, 2012.
13. B. Noia, K. Chakrabarty, and E. J. Marinissen, "Test architecture optimization for TSV based 3D-stacked ICs," in *Proc. of 15th IEEE ETS*, 2010, pp. 24–9.
14. B. Noia, K. Chakrabarty, S. K. Goel, E. J. Marinissen, and J. Verbree, "Test-architecture optimization and test scheduling for TSV-based 3-D stacked ICs," *IEEE Trans. Computer-Aided Des. Integr. Circuits Syst.*, Vol. 30, no. 11, pp. 1705–18, 2011.
15. L. Jiang, L. Huang, Q. Xu, K. Chakrabarty, and T. M. Mak, "Integrated test-architecture optimization and thermal-aware test scheduling for 3-D SoCs under pre-bond test-pin-count constraint," *IEEE Trans. VLSI Syst.*, Vol. 20, no. 9, pp. 1621–33, 2012.
16. S. K. Roy, S. Chatterjee, C. Giri, and H. Rahaman, "Faulty TSVs identification and recovery in 3D stacked ICs during pre-bond testing," in *Proc. of 3DIC*, 2013, pp. 1–4.
17. L. Jiang, and D. Xiang, "Dft optimization for pre-bond testing of 3D-SICs containing TSVs," in *Proc. International Conference on computer design (ICCD)*, 2010, pp. 474–9.
18. L. Jiang, Q. Xu, and B. Eklow, "On effective TSV repair for 3D-stacked ICs," in *Proc. of DATE*, 2012, pp. 793–8.
19. S. K. Roy, P. Ghosh, H. Rahaman, and C. Giri, "Session-based core test scheduling for 3D SOCs," in *Proc. of ISVLSI*, 2014, pp. 196–201.

20. S. K. Roy, and C. Giri, "Design-for-test and test time optimization for 3D SOCs", in *Proc. of ITC*, 2017, pp. 1–10.
21. K. Chakraborty, and E. J. Marinissen, "Test access mechanism optimization, test scheduling, and tester data volume reduction for system-on-chip," *IEEE Trans. Computer*, Vol. 52, no. 12, pp. 1619–32, 2003.
22. S. Banerjee, S. Majumder, and B. B. Bhattacharya, "Test-time reduction for power-aware 3D-SoC," in *Proc. of VLSI Design*, 2018, pp. 103–8.
23. T. Kaibartta, C. Giri, H. Rahaman, and D. K. Das, "Optimizing test time for core-based 3-D," in *Proc. of ASQED*, 2015, pp. 62–7.
24. J. Pouget, E. Larsson, and Z. Peng, "Multiple-constraint driven system-on-chip test time optimization," *J. Electr. Test. Theory Appl. (JETTA)*, Vol. 21, no. 6, pp. 599–611, 2005.

AUTHORS



Surajit Kumar Roy received the BSc (Hons in physics) from Calcutta University, India. He also received Bachelor of Technology in computer science and engineering and subsequently Master of Technology in computer science and engineering from Calcutta University, India, in 2002 and 2004. He was awarded PhD degree from Indian Institute of Engineering Science and Technology (IEST), Shibpur, Howrah, WB India in 2016. Currently, he is working at Indian Institute of Engineering Science and Technology, India, as assistant professor in the department of Information Technology. His research interest includes VLSI testing, embedded systems, and hardware security.

Corresponding author. Email: suraroy@gmail.com



Chandan Giri has been at Indian Institute of Engineering Science and Technology, Shibpur, Howrah, WB, India since 2008 as assistant professor, Department of Information Technology. His current research is focused on testing and design-for-testability of integrated circuits (especially 3D and multicore chips) and wireless sensor network. His research project included 3D multicore IC testing. Research support is provided by the University Grant Commission, Government of India. C Giri received his Bachelor of Technology (BTech) in computer science and engineering from Calcutta University, India, in 2000 and subsequently Masters of Engineering (ME) in computer science and engineering from Jadavpur University, Kolkata, India, in 2002. He was awarded PhD degree from Department of Electronics and Electrical Communication Engineering of Indian Institute of Technology, Kharagpur, in 2008. He also presented his research papers in several international conferences. He is a member of IEEE and ACM.

Email: chandangiri@gmail.com